


# MSXPi Interface

Version 1.6

User's and Developers Guide

https://github.com/costarc/MSXPi

Copyright (c) 2015-2026 Ronivon Costa.

Software: MIT licence. Hardware: CERN OHL v1.2.

# Table of Contents

**1 Introduction**
- 1.1 About this guide
- 1.2 What's new in version 1.6
- 1.3 What MSXPi is and how it works
- 1.4 Hardware compatibility and Raspberry Pi models

**2 Architecture**
- 2.1 The interface and its I/O ports
- 2.2 Software layers
- 2.3 The block transfer protocol
- 2.4 Commands and replies
- 2.5 Server internals
- 2.6 Disk access
- 2.7 The Ethernet UNAPI
- 2.8 Loading ROMs from the network

**3 Getting Started**
- 3.1 Assembling the interface
- 3.2 Programming the CPLD
- 3.3 Writing the ROM to the EEPROM
- 3.4 Preparing the SD card for the Raspberry Pi
- 3.5 The MSXPi home directory and msxpi.ini
- 3.6 Client software on the MSX
- 3.7 Booting with MSXPi
  - 3.7.1 Booting from another interface with Nextor or MSX-DOS 2 (recommended; MSXPi provides no Nextor of its own)
  - 3.7.2 Booting MSX-DOS 1 from the MSXPi ROM
- 3.8 MSXPi under openMSX

**4 Users Guide**
- 4.1 Getting around
- 4.2 The p commands
  - 4.2.1 pver
  - 4.2.2 p cd and p dir
  - 4.2.3 pcopy
  - 4.2.4 p run
  - 4.2.5 p set
  - 4.2.6 p date
  - 4.2.7 p wifi, p wlanreset, p netreset
  - 4.2.8 p play and p vol
  - 4.2.9 p reload, p reboot, p shut, p restart
  - 4.2.10 p chatgpt
  - 4.2.11 p showpage
  - 4.2.12 p music
  - 4.2.13 pchess
- 4.3 msxarch - the network ROM browser
- 4.4 LOADROM
- 4.5 at28c256 - writing the EEPROM
- 4.6 Updating MSXPi
- 4.7 TCP/IP with InterNestor Lite
- 4.8 BASIC programs
- 4.9 Dealing with errors and instabilities

4.9.1 Recovery with CTRL+ESC
4.9.2 Checking what is alive
4.9.3 Failed MSX-DOS 1 boot with a working MSXPi
4.9.4 Failed boot with a failing MSXPi
4.9.5 Older boards

**5 Developers Guide**
5.1 The programming model
5.2 Adding a command to the server
5.3 Tools and build
5.4 Testing under openMSX

**6 Developing in BASIC**
6.1 CALL MSXPI
6.2 The BASIC API: API.BAS
6.3 Tips for BASIC programs

**7 Developing in C**
7.1 A minimal program
7.2 API reference
7.3 Example: pver.c - talk to the hardware and the server
7.4 Example: pcopy.c - a conversation with several blocks
7.5 Example: msxarch.c - reading structured data
7.6 The shared link and the Ethernet UNAPI
7.7 Memory and buffers
7.8 Error handling

**8 Developing in Assembly**
8.1 Skeleton of a .COM program
8.2 BIOS routines
8.3 Example: receiving a reply block by block
8.4 Example: the BASIC CALL handler
8.5 Example: the disk driver
8.6 Other useful sources

**9 Appendices**
9.1 Appendix A: Return codes and protocol constants
9.2 Appendix B: msxpi.ini variables
9.3 Appendix C: GPIO pin assignments
9.4 Appendix D: The MSXPi device for openMSX and the CPLD
9.5 Appendix E: Files in software/target

# 1 Introduction

MSXPi is a bridge between a classic MSX computer and a Raspberry Pi. A cartridge (the MSXPi interface) exposes three I/O ports to the MSX; whatever the MSX writes to them is delivered to a Python program, the **msxpi-server**, and whatever the server answers is read back by the MSX. Through this simple link the MSX gains network drives, the internet, disk images, ROM archives, ChatGPT, IRC, TCP/IP (through the UNAPI standard) and control over the Raspberry Pi itself.

MSXPi is also available as a virtual device in the **openMSX** emulator. The emulated MSX runs exactly the same ROM and the same programs, and talks to the same msxpi-server over a local TCP socket, so software can be developed and tested without any hardware. Software written under openMSX runs unchanged on a real MSX with MSXPi.

## 1.1 About this guide

This guide is for two audiences and is organised accordingly:

- **Users** need chapters 3 (Getting Started) and 4 (Users Guide): how to build or install MSXPi, prepare the Raspberry Pi, use every `p` command, load ROMs, get the MSX on the network and recover from problems.
- **Developers** need chapters 2 (Architecture) and 5 to 8 (Developers Guide, BASIC, C, Assembly): how the pieces fit together, the block protocol, and how to write programs in BASIC, C and Assembly and add commands to the server.
- The appendices hold reference tables: return codes, I/O ports, the msxpi.ini variables and the GPIO wiring.

The README in the repository has a shorter step-by-step Quick Start for both real hardware and openMSX; the Release_Notes file lists what changed in each version.

## 1.2 What's new in version 1.6

Version 1.6 breaks software compatibility with earlier ROMs and servers: upgrade the ROM, the msxpi-server, the client commands and (on real hardware) the CPLD firmware together. The main changes:

- **/WAIT flow control.** The CPLD holds the Z80 /WAIT line while a byte is in flight, so payloads move with INIR/OTIR instead of polling. Measured on real hardware, 256 KB now takes 51.95 s instead of 1:45. A v1.6 ROM on older firmware falls back to the polled transfers and still works, only slower.
- **Ethernet UNAPI in the ROM.** InterNestor Lite, telnet, HGET and other UNAPI programs work over the MSXPi link, on the Pi and in openMSX (also on Windows).
- **MSX-DOS 2 / Nextor coexistence:** fixes to the disk driver (GETDPB, direct sector transfers) so an MSXPi drive works from a Nextor or MSX-DOS 2 system that runs on another interface. The MSXPi ROM itself contains MSX-DOS 1 only.
- **msxarch**, a network ROM browser and loader for plain ROMs and MegaROMs, with server-side mapper detection; `LOADROM /N` loads a ROM by name.

- **New and improved commands:** `pver` (hardware report), `pcopy` in both directions, `p netreset`, `p wlanreset`, `p reload`, a fixed `p wifi`.
- **Older boards:** the v1.6 CPLD image, ROM and server run on every earlier MSXPi board.
- **BASIC extension without an EPROM:** MSXPIEXT.BIN builds again and shares the `CALL MSXPI` handler with the ROM.
- **openMSX** emulates the CPLD /WAIT control and no longer truncates large transfers.

## 1.3 What MSXPi is and how it works

MSXPi combines a custom MSX cartridge with a Raspberry Pi Zero class board. The interface exposes I/O ports that the MSX can read and write, and the Raspberry Pi interprets what arrives to provide services such as:

- access to the Pi SD card and to network shares as storage (`p dir`, `pcopy`);
- WiFi and TCP/IP for the MSX, and internet access (HTTP, HTTPS, FTP, SMB);
- disk images: the Pi can present disk images as drives A: and B: and boot MSX-DOS 1 from them;
- loading ROM games from archives on the network (`msxarch`, `LOADROM /N`);
- ChatGPT, IRC, weather and stock quote clients;
- audio playback on the Pi, and running any shell command on the Pi from MSX-DOS.

## 1.4 Hardware compatibility and Raspberry Pi models

MSXPi works with most MSX generations (MSX1, MSX2, MSX2+, turbo R), but compatibility is not universal: a few models have unusual slot or bus implementations. MSX-DOS itself needs 64 KB of RAM.

- **Raspberry Pi Zero 2 W / Zero W** - recommended: small, built-in WiFi, low power, and powered directly from the cartridge slot.
- **Raspberry Pi Zero (no W)** - compatible, but needs a USB WiFi dongle for networking.
- **Raspberry Pi Pico** - not supported. A Pico is a microcontroller and cannot run the Linux-based server.

The MSXPi PCB expects the Pi with its 40-pin header on a specific side. A Zero WH has the header on the component side; use it only if your PCB has the socket on the back. See section 3.1.

# 2 Architecture

MSXPi has three parts: the **MSX side** (ROM plus programs), the **interface** (CPLD and EEPROM in a cartridge) and the **server side** (msxpi-server.py). The same server code runs on the Raspberry Pi, where the link is five GPIO wires, and on a PC beside openMSX, where the link is a TCP socket.

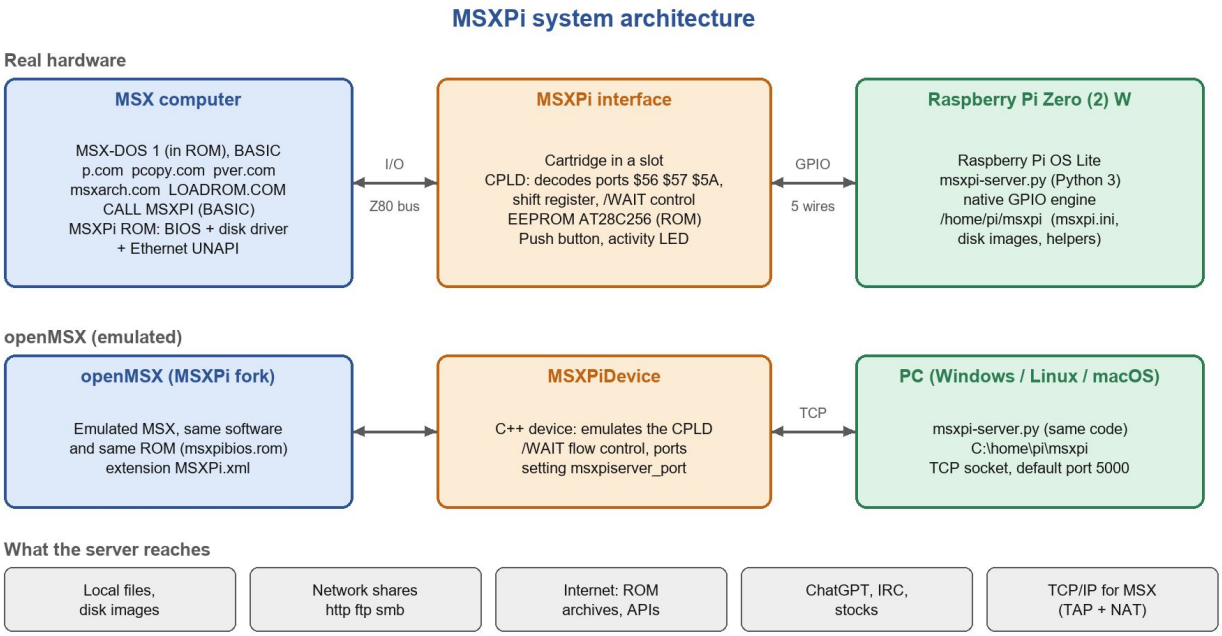



*Figure 1: System architecture: real hardware (top) and openMSX (bottom)*

## 2.1 The interface and its I/O ports

The CPLD decodes three I/O ports and turns every access into an 8-bit serial transfer with the Pi (SPI-like: CS, CLK, MOSI, MISO plus a ready line). The MSX never sees the wires, only the ports:

| Port | Name | Use |
|---|---|---|
| $56 | CONTROL_PORT1 | OUT $FF resets the transfer state and clears /WAIT mode. OUT $00 starts a one-byte transfer. IN reads 0 when no transfer is running and the Pi is ready. |
| $57 | CONTROL_PORT2 | IN returns the CPLD build ID (low 6 bits; $FE/$FF: no interface). OUT $01 switches /WAIT mode on, OUT $00 off. Firmware without the mode register ignores the write, which is how software probes for /WAIT support. |
| $5A | DATA_PORT1 | OUT: byte to send to the Pi (data or command). IN: byte received from the Pi. |

Since v1.6 a byte transfer can also run in **/WAIT mode**: after OUT to $5A or IN from $5A the CPLD pulls the Z80 /WAIT line low until the byte has been clocked, so a block instruction (INIR, OTIR) can move a whole 256 or 512 byte run at 21 T-states per byte. The Pi keeps its ready line high for the entire run (SPI_BurstOut / SPI_BurstIn); without that, the gap between two bytes would let INIR read a stale byte and desynchronise the stream.

The ROM in the EEPROM (`msxpibios.rom`, 16 KB at $4000, header "AB") holds the BIOS, the disk driver and the Ethernet UNAPI. The MSXPi PCBs allow the ROM to be disabled with a jumper, in which

case the interface is used only through the `.COM` programs, or replaced by MSXPIEXT.BIN (loaded into RAM from BASIC) when there is no EEPROM.

## 2.2 Software layers

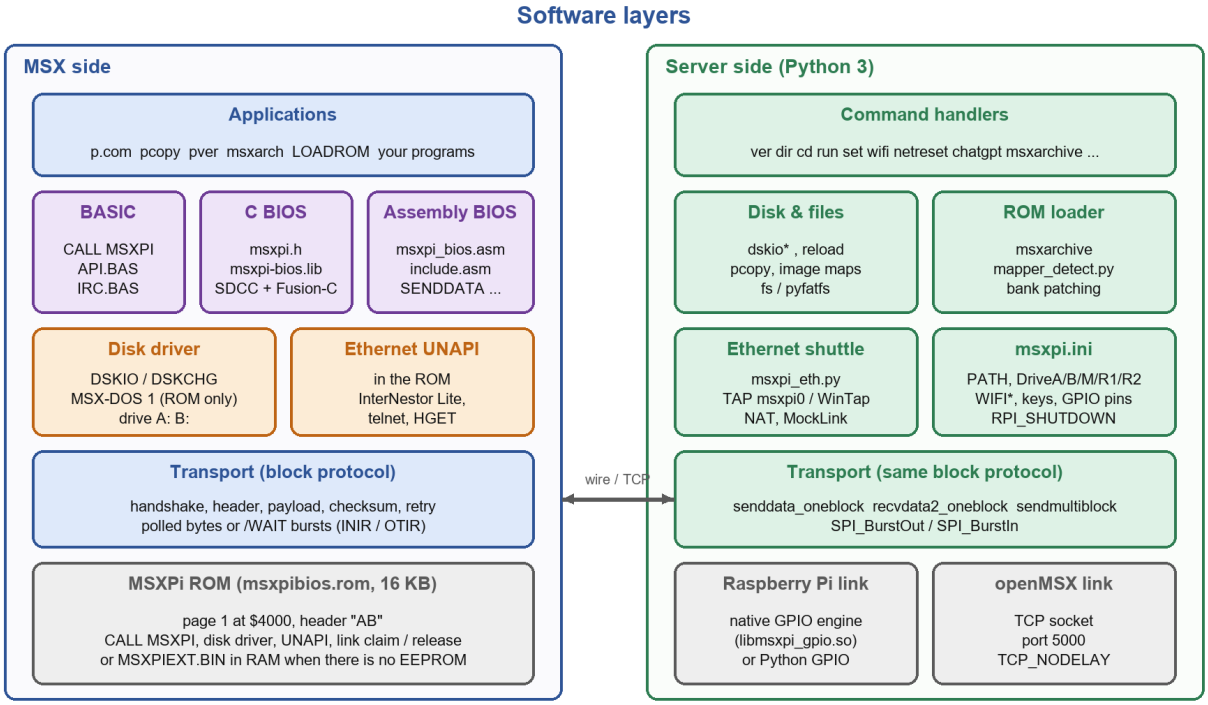



*Figure 2: Software layers on the MSX and on the server*

On the MSX every program - a `.COM` file, a BASIC program, a driver - uses the same transport routines, available in three flavours: `CALL MSXPI` for BASIC, the C BIOS (`msxpi.h`, `msxpi-bios.lib`) and the assembly BIOS (`msxpi_bios.asm`). They all implement the same block protocol of section 2.3.

On the server, msxpi-server.py is a small main loop; the commands live in the msxpi_cmd_*.py modules beside it. It reads a command and calls the Python function registered under the first word in the COMMANDS table; the function sends its reply with sendmultiblock(). Adding a command to MSXPi therefore means writing one Python function and registering it (section 5.2).

## 2.3 The block transfer protocol

Everything the MSX and the server say to each other travels in **blocks**. A block carries one command, one reply, or one chunk of a larger transfer, protected by a checksum:

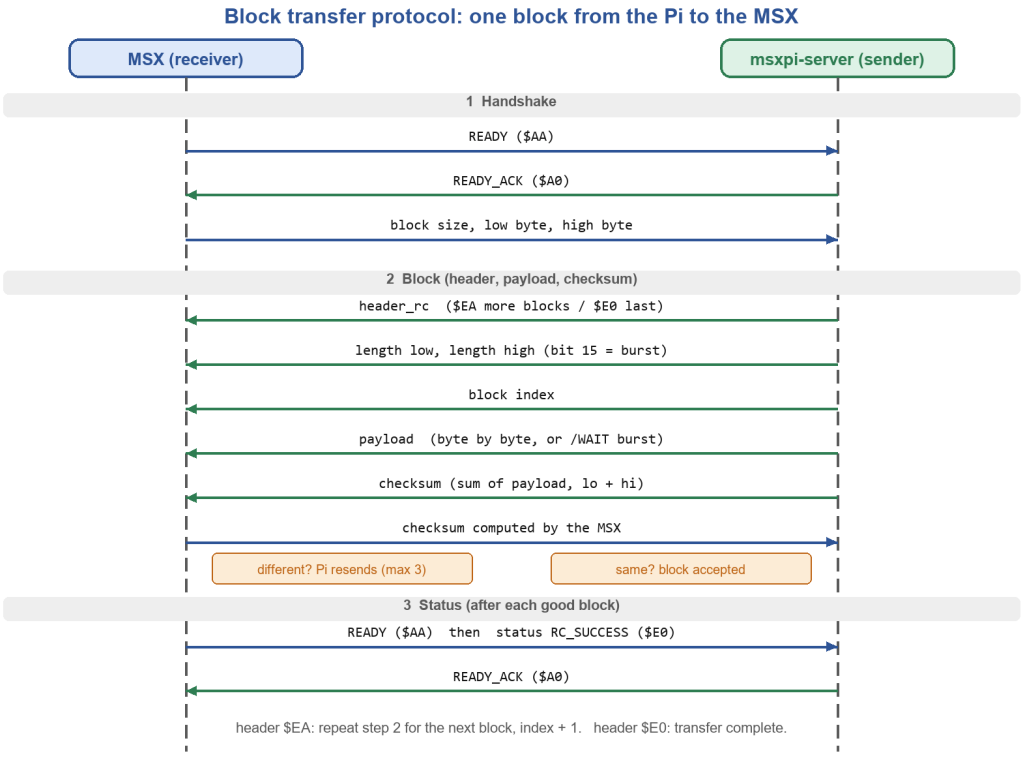



*Figure 3: One block, Pi to MSX. MSX to Pi is symmetrical: the MSX becomes the sender.*

| Field | Size | Meaning |
|---|---|---|
| `header_rc` | 1 | Return code: $EA RC_READY (more blocks follow), $E0 RC_SUCCESS (last block), or an error code |
| `length` | 2 | Payload size, low byte first. Bit 15 set means "payload follows in /WAIT burst mode" |
| `block index` | 1 | Sequence number, starting at 0 |
| `payload` | length | The data. Blocks are at most BLKSIZE (512) bytes on the wire; the receiver announces its maximum in the handshake |
| `checksum` | 1 | Sum of all payload bytes, low byte plus high byte of the 16-bit sum |

Rules worth knowing:

- The receiver echoes the checksum it computed. If it differs, the sender resends the **whole** block (header included), up to three times; then the transfer fails with RC_CHKSUM_ERR. A retry never uses /WAIT bursts.
- Burst is negotiated per block: the receiver marks the block size it asks for, the sender marks the length it answers with, and either side can decline. Bursts are used only for whole 256-byte runs (a 512-byte disk sector); anything else goes byte by byte.
- After every good block there is a short status exchange (READY, status, READY_ACK) so both sides agree whether another block follows.
- While waiting for its counterpart, each side polls the keyboard: **CTRL+ESC** aborts the transfer on the MSX (ESC alone is an ordinary key and never aborts), which makes the server resynchronise (about five seconds).
- Timeouts are generous on the server (180 s per byte wait) because a slow MSX may take a long time to drain a large block.

## 2.4 Commands and replies

A command is a text block. The first word names the server function, the rest are its parameters; for MSX-DOS programs the parameters are the DOS command line. The server replies with one or more blocks:

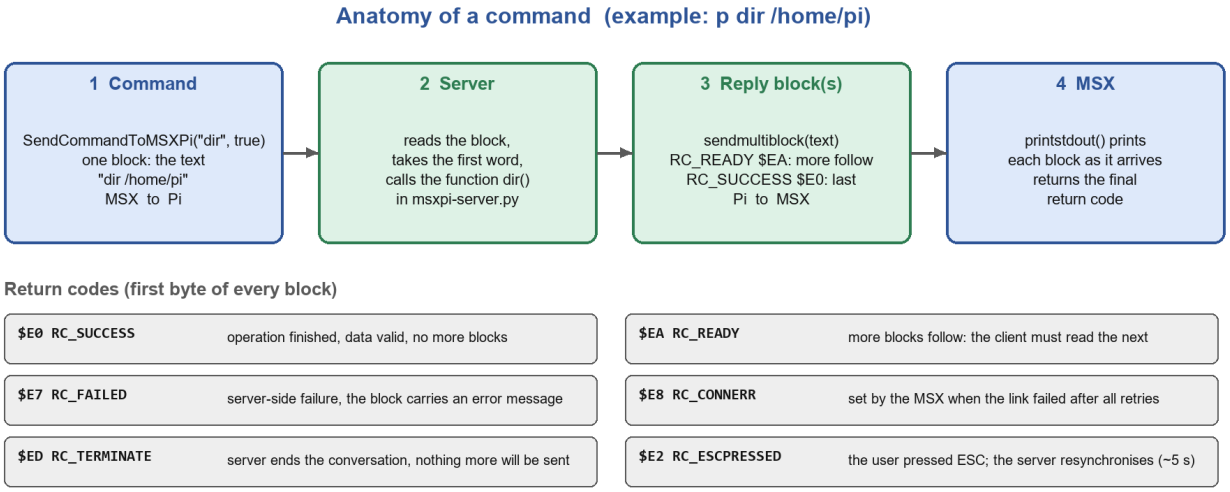



*Figure 4: Life of a command, and the return codes*

Every reply block starts with a return code. The MSX must read blocks until it sees a code other than RC_READY; a client that stops early leaves the server waiting to send, and both sides then need the CTRL+ESC recovery. The full table of codes is in Appendix A.

Most commands follow one of four shapes; the block protocol is the same in all of them:

1. **Command, wait for the reply, print it** - by far the most common: `p dir`, `p ver`, `p run`. This is what CALL MSXPI, `printstdout()` and PRINTPISTDOUT implement.
2. **Command only**, no reply needed (rare).
3. **Command plus extra parameter blocks**, then a reply - for example `p set` with a long value.
4. **A conversation**: several commands or blocks in both directions while the server keeps the state. `pcopy` (init, readblock, writeblock, putclose), `msxarch` (list, select, header, image blocks) and the disk driver (DSKIO reads and writes) work this way. IRC.BAS keeps its state on the server between separate CALL MSXPI calls.

## 2.5 Server internals

- Main loop. Waits for a command block, splits it, looks up COMMANDS[cmd.lower()] (an explicit table in msxpi-server.py - only the functions listed there can be called from the MSX) and calls it with the parameter string. If the function returns a string or bytes, the loop sends it as the reply; unknown commands answer "Unknown command"; an exception answers "Pi:Error - ..." and keeps the server alive.
- **Transport.** On a Raspberry Pi the bytes travel over GPIO (pins CS, SCLK, MOSI, MISO, RPI_READY, set in msxpi.ini). A native C engine (`native/libmsxpi_gpio.so`, enabled with MSXPI_NATIVE_GPIO=1 and on by default in msxpi-monitor and the systemd unit) clocks the

payload; without it the server falls back to Python GPIO and says so in its log. Under openMSX the same routines read and write a TCP socket instead (port 5000, TCP_NODELAY).

- **Configuration.** `msxpi.ini` in the MSXPi home directory is read at start-up and rewritten by `p set`. Paths default to `/home/pi/msxpi`, which is hard-coded; on Windows it resolves on the current drive (C:\home\pi\msxpi).
- **Disk images.** DriveA and DriveB point at `.dsk` files that the server memory-maps, so writes from the MSX persist. `p reload A:` re-opens an image after it was replaced.
- **Ethernet shuttle** (`msxpi_eth.py`): carries UNAPI Ethernet frames between the MSX link and a TAP device (section 2.7).
- **Service.** On the Pi, `msxpi-monitor` (a systemd unit) starts, monitors and restarts the server, and runs the TCP/IP setup script in the background.

## 2.6 Disk access

The ROM contains **MSX-DOS 1 only** (the 1.03 kernel and its disk driver); Nextor and MSX-DOS 2 are not part of MSXPi and must come from another disk interface. The driver is written so that it also works when Nextor is the system in charge. Drives A: and B: of the MSXPi map to the disk images named by DriveA and DriveB in msxpi.ini (by default `msxpiboot.dsk` and `tools.dsk`). Each sector is requested from the server (`dskior`, `dskiow`) and moved by /WAIT bursts when both sides negotiated them.

The driver transfers directly to the caller's buffer when that is outside page 1, and through the kernel's XFER routine when it is in page 1, because page 1 is where the driver ROM itself is banked in. A drive mapped with MAPDRV under Nextor (running from the other interface) works the same way; GETDPB clears carry explicitly, which is what Nextor needs to accept the DPB. At the time of writing, disk reads use bursts and writes use the polled path.

## 2.7 The Ethernet UNAPI

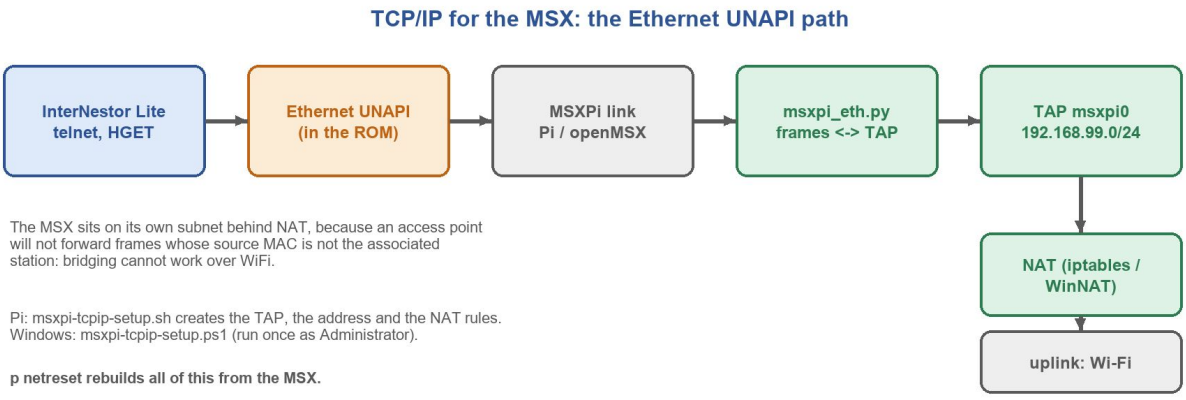



*Figure 5: From an UNAPI program to the internet*

The ROM registers an ETHERNET UNAPI implementation, so programs such as InterNestor Lite (INL) need no extra driver: `ETHUNAPI.COM` refuses to load because the implementation already exists, and `RAMHELPR` must not be run. Frames travel over the same MSXPi link (using the same /WAIT burst code as the disk driver), and a shared-link guard (`msxpi_link_claim` / `msxpi_link_release`) prevents the UNAPI interrupt routine from transmitting in the middle of a command exchange.

On the Pi, `msxpi-tcpip-setup.sh` creates the TAP device `msxpi0`, gives it the address 192.168.99.1/24, enables forwarding and installs the NAT rules for whichever interface carries the default route. The MSX is placed on its own subnet behind NAT because WiFi access points do not forward frames whose source MAC is not the associated station, so bridging cannot work. Under openMSX on Windows the same driver reaches a real network through an OpenVPN TAP-Windows adapter, and `msxpi-tcpip-setup.ps1` (run once, as Administrator) configures the address and a WinNAT instance. If the TAP is not ready when the first UNAPI call arrives, the server retries and swaps it in later; if it never appears the server falls back to "MockLink", which answers every call but carries no traffic.

## 2.8 Loading ROMs from the network

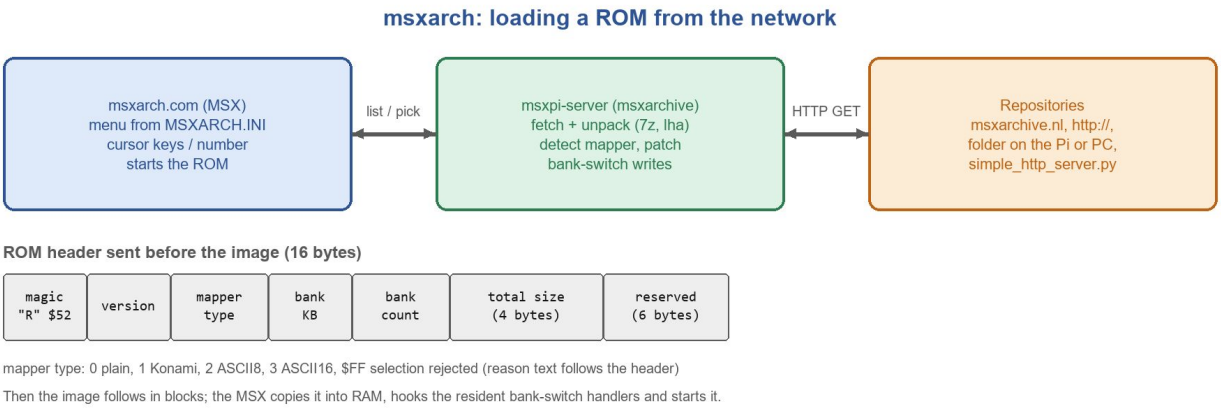



*Figure 6: msxarch and the ROM header*

`msxarch` shows a list of ROMs found in the repositories listed in `MSXARCH.INI` and starts the one you choose. The server downloads and unpacks the archive (into a clean directory, so ROMs from different downloads never mix), classifies the ROM - plain, Konami, Konami SCC, ASCII8 or ASCII16, using openMSX's `softwaredb.xml` when available - patches the bank-switch writes to work with the resident handlers, and sends a 16-byte ROM header followed by the image. If the selection cannot be loaded, the header carries mapper type $FF and a reason text, and the MSX shows it and returns to the list. `LOADROM.COM` (a binary patch of the third-party LOADROM 1.97) does the same for a file name given on the command line: `LOADROM name.rom /N`.

# 3 Getting Started

MSXPi can be used with or without the EEPROM installed. The most practical setup is to use it together with another interface that runs Nextor or MSX-DOS 2 (for example a MegaFlashROM SCC+ SD), with MSXPi providing the Raspberry Pi control and the network. MSXPi itself has only MSX-DOS 1 built into its ROM: it coexists with Nextor and MSX-DOS 2 but does not provide them. Booting MSX-DOS 1 from a disk image on the Pi is also possible but slower.

Throughout this guide, "P commands" are the programs in `software/target` (p.com, pcopy.com, pver.com, msxarch.com, LOADROM.COM, at28c256.com, ...). Copy them to your MSX disk or SD card first.

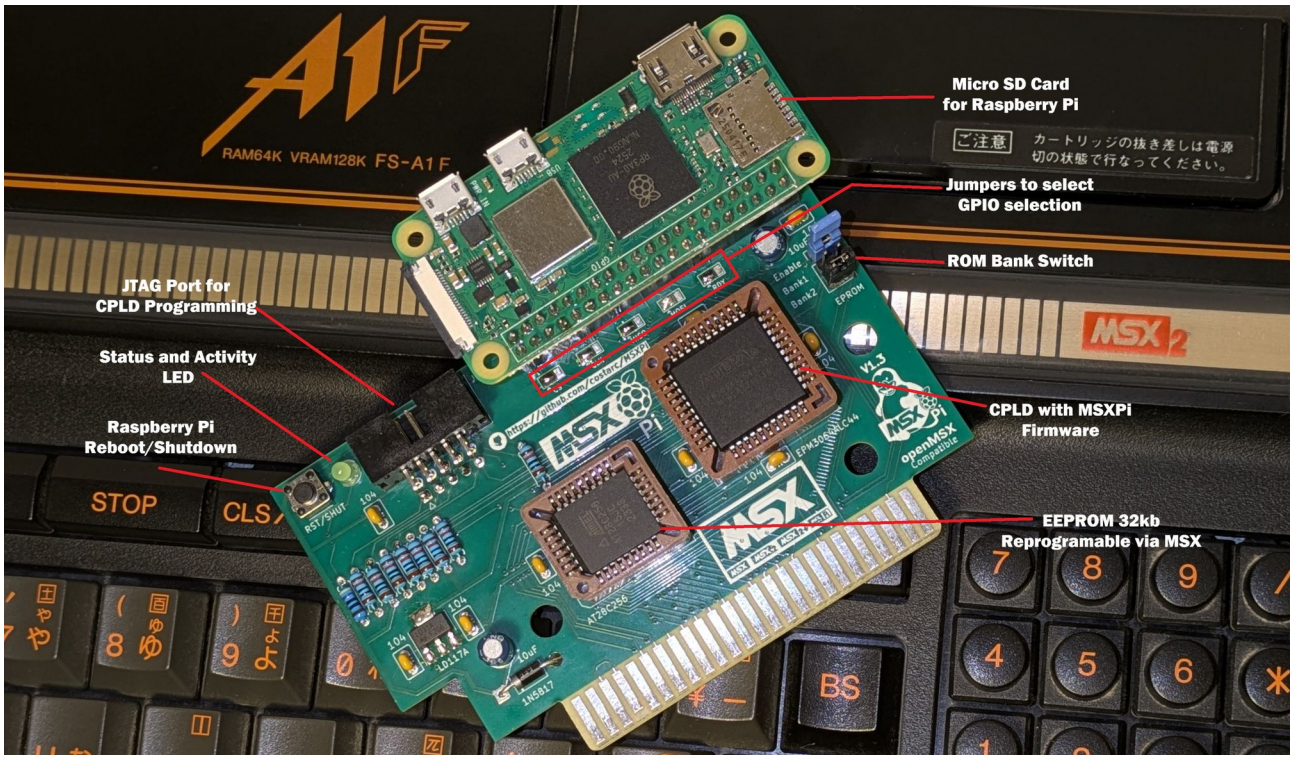



*Figure 7: The main components of the MSXPi interface*

- **Raspberry Pi Zero (2) W** - boots Linux and runs msxpi-server.py.
- **Micro SD card** - holds the Pi's system and the MSXPi home directory.
- **EEPROM AT28C256** - the MSXPi ROM. It is written from MSX-DOS with `at28c256.com`.
- **Jumpers** - enable/disable the ROM (SLTSL) and select the 16 KB bank (A14/A15).
- **CPLD** - decodes the ports and shifts the data. Programmed over the JTAG connector with a USB-Blaster and Quartus on a PC, with the MSX switched on.
- **Activity LED** - lights when the Pi's server is running.
- **Push button** - a short press reboots the Pi; holding it for more than three seconds shuts it down.

## 3.1 Assembling the interface

Assembly order and the CPLD programming steps are in `documents/MSXPi Assembly Instructions.txt`; in short: solder the voltage regulator, the SMD jumpers CS, CLK, MISO, MOSI and RDY, the resistors, the sockets (CPLD, EEPROM, Pi header, JTAG), the capacitors and the remaining parts. Use a socket for the EEPROM so you can replace it later.

The Raspberry Pi's 40-pin header must be on the side that matches your PCB. The Pi is always plugged in with its **component side facing you**. A Zero WH has the header soldered on the front and cannot be used on a PCB whose socket is on the front; use a Zero or Zero W and solder a 2x20 header on the back, as shown below.

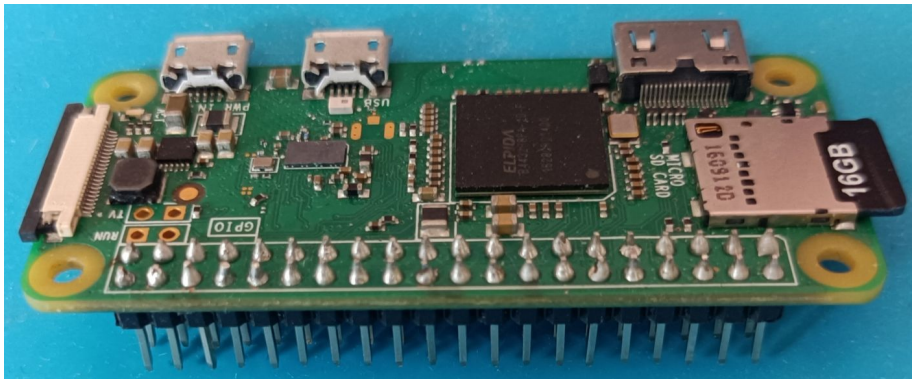



*Figure 8: Soldering the header - top view*

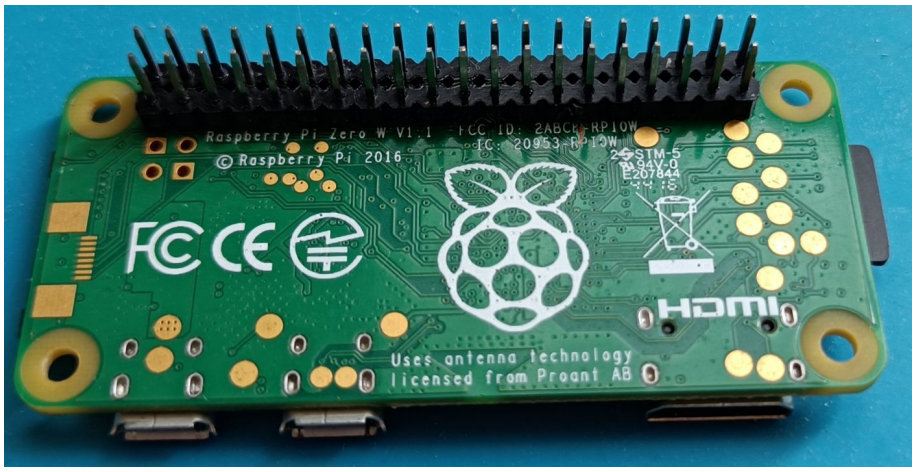



*Figure 9: Soldering the header - bottom view*

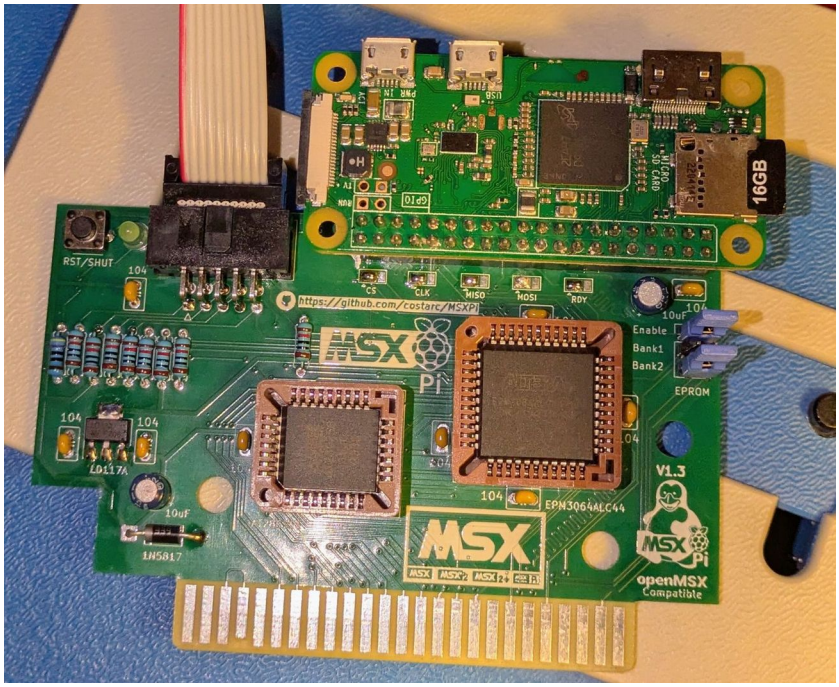



*Figure 10: MSXPi assembled - top view*

## 3.2 Programming the CPLD

Connect a USB-Blaster to the JTAG connector and to the PC, insert the interface in the MSX and switch it on. In Quartus 13.0 SP1: Tools, Programmer, mode JTAG, Hardware Setup shows the USB-Blaster; Add File; tick Program/Configure and Verify; Start. Choose the `.pof` for your board:

| Board | File in hardware/CPLD_Project | Build ID |
|---|---|---|
| V1.3 Rev.1 (and boards for v1.3 to v1.5) | MSXPi_v1.6.pof | $0E |
| V1.1 Rev.1, V1.2.1b | v1.1/MSXPi_v1.6_wait.pof | $14 |
| v0.8.2 / v1.0 (PCB v0.7 Rev.7) | v0.8.2/MSXPi_v1.6_polled.pof | $10 |

Check with `PRINT INP(&H57)` in BASIC (not 254/255) or with `pver`. Older boards need extra settings in msxpi.ini and, for the v0.8.2 board, a specific EPROM jumper setting: see `hardware/CPLD_Project/LEGACY_BOARDS.md` and `documents/Legacy Support.odt`.

## 3.3 Writing the ROM to the EEPROM

1. Copy `at28c256.com` and `msxpibios.rom` (from `software/target`) to a disk of an MSX that has the interface plugged in.
2. If the interface already holds a bootable ROM, remove the enable jumper (SLTSL, or CS12 on older boards) first.
3. List the slots and ROMs: `at28c256 /i`. Note the slot number of the interface.
4. Write: `at28c256 /s 2 msxpibios.rom` (2 is the slot). The tool write-protects the EEPROM when it finishes. Add `/r` to write slowly if the chip fails.
5. Power off, set the jumpers (below) and power on. `CALL MSXPIVER` in BASIC prints the ROM version.

The EEPROM has 32 KB. Two 16 KB ROMs can live in it and be selected with the A14/A15 bank jumper (merge them into one 32 KB file first). The ROM is 16 KB, one bank.

## 3.4 Preparing the SD card for the Raspberry Pi

1. Write "Raspberry Pi OS Lite (32-bit)" with Raspberry Pi Imager (CHOOSE OS, Raspberry Pi OS (other), Raspberry Pi OS LITE). In its settings use the user name `pi`, enable SSH and enter your WiFi.
2. Boot the Pi and log in (SSH, or keyboard and HDMI).
3. Run the setup script: wget https://tinyurl.com/MSXPi-Setup, chmod 755 MSXPi-Setup, bash ./MSXPi-Setup. It installs the Python modules and packages, creates the MSXPi home directory, downloads the server, ini templates and disk images, builds the native GPIO engine, installs the msxpi-monitor service and reboots - a few minutes. It asks which board you have (or pass --board v1.3, v1.1 or old); --help lists the other options. It is safe to run again and never overwrites msxpi.ini. Insert the card in the Pi and plug it into the interface.

The Hard Way tutorial in the documents folder (English and Portuguese) shows the Raspberry Pi OS installation with pictures; the setup script replaces its manual steps. A ready-made SD card image is no longer offered.

## 3.5 The MSXPi home directory and msxpi.ini

Everything the server needs is in one folder, /home/pi/msxpi (on Windows, C:\home\pi\msxpi). The path is built into the server; the environment variable MSXPI_HOME overrides it:

| Item | Purpose |
|---|---|
| `msxpi-server.py, msxpi_*.py (msxpi_cmd_*.py, msxpi_transport.py, msxpi_settings.py, msxpi_eth.py, msxpi_gpio_native.py, ...), mapper_detect.py` | The server and its modules |
| `native/libmsxpi_gpio.so` | Native GPIO engine, built on the Pi by native/build.sh |
| `msxpi.ini` | Your configuration (Appendix B lists every variable) |
| `disks/msxpiboot.dsk, disks/tools.dsk` | Drives A: and B: for the MSX-DOS 1 boot mode |
| `msxpi-monitor, msxpi-tcpip-setup.sh, update.sh` | Service wrapper, network setup and updater |

Create `msxpi.ini` from the template that matches your PCB (`msxpi-JumperLeft.ini`, `msxpi-JumperRight.ini` or `msxpi-JumperRight_PCBV1.1Rev.0.ini` in `software/Server/Python/src`). Boards without the shutdown push-button must add `var RPI_SHUTDOWN=none`. Change values from the MSX with `p set NAME value`.

## 3.6 Client software on the MSX

Copy the files from `software/target` to your MSX disk or SD card: `p.com`, `pcopy.com`, `pver.com`, `msxarch.com`, `msxarch.ini`, `LOADROM.COM`, `at28c256.com`, `INL.CFG`, the telnet clients and the BASIC programs (API.BAS, IRC.BAS, WEATHER.BAS, DOLAR.BAS, STOCKS.BAS). The updater

`msxpiupd.bat` does this over the network. Then type `pver`: it prints the board, the CPLD build and whether /WAIT is available, followed by the server version.

## 3.7 Booting with MSXPi

### 3.7.1 Booting from another interface with Nextor or MSX-DOS 2 (recommended; MSXPi provides no Nextor of its own)

1. Remove the SLTSL (enable) jumper if you do not need the MSXPi ROM, or put MSXPi in a higher slot than the disk interface.
2. Plug MSXPi in with the prepared SD card in the Pi and switch the MSX on. The MSX boots from the disk interface at full speed; the Pi finishes booting in the background.
3. Check with `pver`. If the MSX seems unresponsive, press CTRL+ESC, wait five seconds and try again.

### 3.7.2 Booting MSX-DOS 1 from the MSXPi ROM

1. Make sure the EEPROM holds the current `msxpibios.rom`.
2. Close the SLTSL jumper and select the bank that holds the MSXPi ROM.
3. Switch on and wait until the MSX-DOS 1.03 banner appears. The Pi must complete its boot first, so the first cold boot takes a few minutes.
4. Drives A: and B: are the disk images of DriveA and DriveB. Do not delete files from drive A: in this mode.

Hold ESC during boot to skip the MSXPi boot and go to BASIC.

## 3.8 MSXPi under openMSX

The steps are in the README ("Quick Start 2"). In short: download openMSX from the MSXPi fork releases (https://github.com/costarc/openMSX/releases; the upstream project is https://github.com/openMSX/openMSX), install Python 3 with requests, fs and "setuptools<81", and 7-Zip, create the MSXPi home directory with msxpi-server.py, mapper_detect.py and every msxpi_*.py module, start python msxpi-server.py, then openmsx -machine Panasonic_FS-A1WSX -ext MSXPi -diska FloppyA. On Windows msxpi-windows-setup.ps1 does all of it. The optional openMSX setting msxpiserver_port (default 5000) changes the TCP port; start the server with the environment variable MSXPI_PORT set to the same value. Appendix D compares the openMSX device with the CPLD.

# 4 Users Guide

MSXPi can be used as a network and Raspberry Pi companion that coexists with a Nextor or MSX-DOS 2 system provided by another interface (the recommended way; MSXPi's own ROM has only MSX-DOS 1), or as a stand-alone floppy drive that boots MSX-DOS 1.03 from a disk image on the Pi. In both cases the `p` commands work the same, under MSX-DOS 1, MSX-DOS 2 and Nextor, and CALL MSXPI works in BASIC however the machine was booted.

## 4.1 Getting around

- **CTRL+ESC** interrupts a running command. The server then abandons the operation and re-enters command mode; wait about five seconds before the next command.
- **pver** (or `p ver`) tells you if MSXPi is alive. From BASIC, `CALL MSXPIVER` shows the ROM version and `CALL MSXPI("1,C000,ver")` asks the server.
- **Always run `p shut` before switching the MSX off.** The Pi is powered by the MSX; a sudden power cut can corrupt its SD card.
- A short press of the push button reboots the Pi; holding it for more than three seconds shuts it down.
- Type `p` (or `p /help`) to list the commands, and `p /m` to see the memory the command can use.

## 4.2 The p commands

`p.com` is one program that sends its command line to the server: `p dir /home/pi` sends the text "dir /home/pi". Any function of the server can therefore be called from the MSX. The built-in help lists these:

| Command | Description |
|---|---|
| `p ver` | Server version and build |
| `p cd, p dir` | Set the current path and list it |
| `p run <command>` | Run a command on the Pi |
| `p date` | Set the MSX date and time from the Pi |
| `p set` | Show or change MSXPi variables |
| `p wifi` | Show interfaces and configure WiFi |
| `p wlanreset [secs]` | Reset wlan0 and wait for an address |
| `p netreset [secs] (p tcpip)` | Rebuild the Pi's TCP/IP setup for the UNAPI |
| `p vol, p play` | Volume and audio playback on the Pi |
| `p reload A: \| B:` | Reload a drive's disk image |
| `p reboot, p shut, p restart` | Reboot the Pi, shut it down, restart the server |
| `p chatgpt <text>` | Ask ChatGPT |

Besides `p`, these programs come with MSXPi: `pver`, `pcopy`, `msxarch`, `LOADROM`, `at28c256`, `msxpiupd.bat`, the BASIC programs and the UNAPI tools. They are described one by one in the sections below.

### 4.2.1 pver

Reports the hardware and the server. Every line fits 29 columns, the width of MSX-DOS 1 in SCREEN 1:

```
A>pver
 Port $57: $0E
 Build ID: $0E
 Board   : v1.3 to v1.6
 CPLD    : v1.6 /WAIT build
 /WAIT   : yes (burst)

MSXPi Server Version 1.6 Build 20260919.060
```

`/WAIT: no (polled)` means the CPLD firmware or the emulator has no /WAIT mode and the slower polled transfers are used. "Board: no MSXPi found" means no CPLD answered on port $57. "Server: connection error" means the interface is present but the Pi is not answering (still booting, server stopped, or mismatched versions).

### 4.2.2 p cd and p dir

`p cd` sets the path used by `p dir` and `pcopy`; alone it displays it. A path can be on the Pi, a network URL (http, https, ftp, smb) or one of the virtual drives kept in msxpi.ini:

| Drive | Default | Meaning |
|---|---|---|
| `m:` | ftp://192.168.1.100 | Your local network server |
| `r1:` | msxarchive.nl MSX1 ROMs | Internet location |
| `r2:` | msxarchive.nl MSX2 ROMs | Internet location |

```
p cd
 p cd /home/pi
 p cd r1:
 p dir
 p dir /home/pi/msxpi
p dir http://www.msxarchive.nl/pub/msx
```

### 4.2.3 pcopy

Copies files between the Pi (or the network) and the MSX drive, in either direction:

```
pcopy <pi file> [drive:][target]      Pi -> MSX
 pcopy <drive>:<file> <pi file>         MSX -> Pi
pcopy /z r1:frogger.zip               download, unpack, save the ROM
```

Examples: `pcopy /home/pi/msxpi/msxpi.ini msxpi.ini`, `pcopy m:pver.com`, `pcopy b:report.txt /home/pi/report.txt`. With `/z` the file is unpacked (zip, lzh, pma, 7z and others) and the unpacked file is saved. Large files move in 16 KB blocks; a damaged block is retried, a failed upload never destroys the target file, and copying to a drive whose image is on the Pi is done on the Pi side.

### 4.2.4 p run

Runs a command on the Pi and prints its output. Commands that ask for input are not supported. Replace a pipe with `::`:

```
p run ls -l /etc
 p run cat /home/pi/msxpi/msxpi.ini
 p run ps -ef :: grep msx
```

```
p run wget http://www.msxarchive.nl/pub/msx/games/roms/msx1/frogger.zip
```

### 4.2.5 p set

Shows or changes the variables kept in `/home/pi/msxpi/msxpi.ini` (see Appendix B). When a disk image variable changes, the new image is loaded at once.

```
p set
  p set WIFISSID My Wifi Name
  p set WIFIPWD MyWifiPass
  p set OPENAIKEY sk-...
p set DriveB /home/pi/msxpi/disks/other.dsk
```

> **Note:** Do not change DriveA unless you know what you are doing: it holds the MSX-DOS 1 boot disk, and a wrong value can lock you out of the DOS until you edit msxpi.ini by hand.

### 4.2.6 p date

Sets the MSX-DOS date and time from the Pi. It takes no parameters. Put it in AUTOEXEC.BAT.

### 4.2.7 p wifi, p wlanreset, p netreset

`p wifi` lists every network interface with its state and address, one line per address (40 columns). `p wifi set` configures the WiFi from the WIFISSID, WIFIPWD and WIFICOUNTRY variables; reboot the Pi afterwards (`p reboot`).

`p wlanreset [secs]` resets wlan0 without a reboot: it drops the lease, cycles the radio, asks NetworkManager, dhcpcd or dhclient (whichever owns the interface) to reconnect, and waits for a fresh IPv4 address (30 s by default, 5 to 90).

`p netreset [secs]` (alias `p tcpip`) rebuilds the whole TCP/IP setup for the MSX: it tears down the TAP device and the NAT rules, runs `msxpi-tcpip-setup.sh` again and reopens the link. Use it when the Pi had no default route at boot, so the MSX has no network. The optional argument is the time to wait for a default route (15 s by default, at most 60). Run it after `p wlanreset` if the uplink address changed.

### 4.2.8 p play and p vol

Play audio on the Pi (Raspberry Pi only). A unique id is printed when a file starts; use it with pause, resume and stop.

```
p play play Aleste.mp3
  1465
  p play pause 1465
  p play resume 1465
  p play stop 1465
  p play loop Aleste.mp3
  p play getids
  p play getlids
p vol 80%
```

### 4.2.9 p reload, p reboot, p shut, p restart

`p reload A:` (or `B:`) re-opens the disk image after you replaced it on the Pi. `p reboot` reboots the Pi, `p shut` shuts it down cleanly, and `p restart` restarts only the msxpi-server.

### 4.2.10 p chatgpt

Asks ChatGPT and prints the answer. It needs an OpenAI key (`p set OPENAIKEY ...`); the model comes from OPENAIMODEL (default gpt-4o-mini).

```
p chatgpt Write a short BASIC program that draws a circle in SCREEN 2
```

### 4.2.11 p showpage

p showpage [/4|/6|/8] <url> renders a web page on the MSX screen. The page is rendered on the Pi and sent as an image; /4, /6 and /8 pick SCREEN 4, 6 or 8 (MSX2).

### 4.2.12 p music

p music plays audio files on the Pi (Raspberry Pi only), like p play; p vol sets the volume.

### 4.2.13 pchess

pchess.com is a chess game for any MSX2 (SCREEN 5, 64 KB VRAM is enough); pchess1.com is the MSX1 version. By default the MSXPi server validates every move with the optional Python package chess (pip install -r requirements-pchess.txt) and runs the Stockfish AI. Modes: two players on one MSX, against the AI, in a room (an HTTP relay on the local network, or an IRC channel with PCHESSROOMS=irc) and online over IRC (#msxpi). LINK in the ESC menu switches to a TCP/IP UNAPI stack instead (MSR I or RAMHELPR I, then INL I): the MSX then checks the rules itself and plays local, room and online games without the MSXPi server, configured by PCHESS.INI. See the PChess Users Manual, software/Client/src/PCHESS.md and documents/PCHESS-IRC.md.

## 4.3 msxarch - the network ROM browser

`msxarch` lists ROMs from one or more repositories and starts the one you pick. It works under MSX-DOS 1, MSX-DOS 2 and Nextor and supports plain ROMs and MegaROMs (Konami, Konami SCC, ASCII8, ASCII16).

The repositories are read from `MSXARCH.INI`, one per line. Lines starting with `;` or `#` are comments:

```
; msxarch.ini
rebootAfterRomLoad=yes
https://www.msxarchive.nl/pub/msx/games/roms/msx1
https://www.msxarchive.nl/pub/msx/games/roms/msx2
http://localhost:8000
/home/pi/msxpi/gameroms
```

An entry can be an http or https listing (a web server, or `simple_http_server.py` from `software/Server/Python/src` to publish a folder of your PC), or a folder on the Pi (or on the PC running openMSX). Pick a repository from the menu, then in the game list use **N** or cursor down for the next page, **P** or cursor up for the previous page, type a game number and press Enter to load it (Backspace edits the number), and **Q** to quit. The list shows the size of each file, and a progress display shows the loading. Archives are unpacked on the Pi. If a ROM cannot be loaded, the reason is shown and you return to the list.

`rebootAfterRomLoad=yes` resets the MSX after loading so that the game starts on a clean machine. msxarch needs free RAM (a MegaROM is held in mapped RAM segments), so run it from a clean boot.

## 4.4 LOADROM

`LOADROM.COM` is a binary patch of the third-party LOADROM 1.97 (by Victor Martinez and Trunks) with MSXPi network loading added. Without `/N` it behaves exactly like the original and loads from disk; with `/N` the file name is resolved by the server:

```
LOADROM USAS.ROM /N
```

See `software/Client/src/loadrom/README.md` for how the patch works and why.

## 4.5 at28c256 - writing the EEPROM

Writes a ROM file to the interface EEPROM (8, 16 or 32 KB) from MSX-DOS. See section 3.3 for the whole procedure.

```
at28c256 /i                       list identified ROMs and slots
at28c256 /s 2 msxpibios.rom       write to the interface in slot 2
at28c256 /s 2 /r msxpibios.rom    same, writing slowly
```

## 4.6 Updating MSXPi

Updating over the network is risky: keep a keyboard and a monitor at hand, or SSH access to the Pi, in case an update fails.

1. **MSX side:** run `msxpiupd.bat` (needs a working network). It refreshes itself, sets the date, and copies every `.COM`, the BASIC programs and the disk images from the repository. On MSX-DOS 1 the ECHO lines print harmless errors. If you have a very old updater, replace it first: `pcopy https://github.com/costarc/MSXPi/raw/master/software/target/msxpiupd.bat`.
2. **Server side:** from the MSX, `p run wget https://tinyurl.com/MSXPi-Setup`, `p run chmod 755 MSXPi-Setup`, `p run sudo ./MSXPi-Setup`. The MSX prompt stays locked (up to five minutes) until the Pi reboots. Check with `pver`.
3. **ROM:** the updater downloads the new `msxpibios.rom` but never writes it. Write it with `at28c256` (section 3.3).
4. **CPLD:** a new firmware (`.pof`) has to be programmed from a PC (section 3.2).

## 4.7 TCP/IP with InterNestor Lite

With the Ethernet UNAPI in the ROM, no driver has to be installed. Make sure the Pi has a network (`p wifi`), then:

```
MSR I           (MSX-DOS 1 only)
INL I
HOST GOOGLE.COM
TELNET some.host
```

- Do not run `RAMHELPR I`: it installs the UNAPI RAM helper on its own and MSR then finds a helper already present and aborts, leaving INL to fail with "No mapper support routines found". A cold boot clears it. Under Nextor or MSX-DOS 2 skip MSR entirely; the mapper routines are already there.
- `ETHUNAPI` refuses to install because the ROM already registers an ETHERNET implementation.

- If `HOST` says "8: DNS not found" the resolver is the problem, not the link. INL.CFG ships with 1.1.1.1; some networks block public resolvers. Use `INL IP P <address>` with the resolver the Pi reports (`p netreset` prints it).
- If the network never comes up, run `p netreset`. The utilities `ETHTEST`, `ETHMODE`, `ETHBENCH` and `UCOUNT` in `software/UNAPI/bin` test and measure the link.

## 4.8 Online games: PLAYNET and MCHAT

**Alpha.** Online play is experimental: it has been tested between emulators and on the bench, not yet on a wide range of real machines. Expect rough edges, and update the ROM, the server and the clients together.

`MCHAT` is the chat. Type `/play <nick> <game>` to invite another player and `/accept` to take an invitation; both machines then start the game, `PLAYNET` runs it in lockstep with the other machine, and when the game ends (or either player presses Ctrl+Shift+F5) you are back in the chat. The games are Konami's Soccer and Ping Pong, Salamander and Yie Ar Kung-Fu.

The game's two machines can be connected in three ways. `MCHAT` picks the first it finds: through the MSXPi when there is one, over TCP/IP otherwise. `MCHAT /T` forces TCP/IP even when there is an MSXPi.

- **Through the MSXPi.** `MCHAT` with no option on a machine with an MSXPi. The chat and the game both run through the Pi; the Pi talks to the other player. Any MSX with 64 KB works.
- **TCP/IP chat, game through the MSXPi.** `MCHAT /T` on a machine with an MSXPi. The chat and the TCP/IP stack stay on TCP/IP; the game's packets go through the MSXPi (this is the faster way to play with a stack in RAM).
- **All over TCP/IP.** `MCHAT /T` on a machine without an MSXPi (GR8NET, DenYoNet or another UNAPI stack), or on one with an MSXPi when `MCHAT.INI` contains `GAMELINK=1`. The game's packets are UDP datagrams sent by the stack.

Players on different links can play each other. Both must have the same game package (the same `.GPF` file, built from your own ROM dump); if the versions differ, or the package needs a newer `PLAYNET`, the chat says the game is incompatible and tells you what to update.

### 4.8.1 What you need over TCP/IP

- The files in one directory: `MCHAT.COM`, `PLAYNET.COM`, `PLAYNET.OVL` and the game's `.GPF`. `MCHAT.INI` is optional. It may contain `IRCADDR`, `IRCPORT`, `IRCNICK`, `MCHATROOM`, `GAMEPORT`, `GAMEADDR` and `GAMELINK=1`, one `KEY=VALUE` per line.
- A TCP/IP UNAPI implementation. With the MSXPi's Ethernet that is InterNestor Lite: under MSX-DOS 1 run `MSR I` and then `INL I`; under Nextor run `RAMHELPR I` and `INL I`. A GR8NET or similar needs no INL.
- A memory mapper (MSR or Nextor provides the routines). An MSX1 with no mapper cannot play over TCP/IP: use an MSXPi. GR8NET's mapper mode 8 (`CALL NETSETMAP(8,M)`) should give MSX-DOS 1 one, but has not been tried yet.
- Salamander also needs a mapper of 1 MB or more under Nextor. On a machine with its own disk drive, booted into MSX-DOS 1, there is not enough free memory for it: `PLAYNET` says so

and asks you to restart the MSX holding CTRL (that frees 1.5 KB) or to use Nextor. Soccer, Ping Pong and Yie Ar fit everywhere.

- A way for the other player to reach you: the games connect directly (the chat's server helps the two machines find each other, STUN). Behind a symmetric or carrier-grade NAT this fails, and both players are told so.

### 4.8.2 What happens to the chat

Over TCP/IP, `MCHAT` leaves IRC (`QUIT`) just before the game starts and connects again when you are back: an open chat connection slows the game down by up to a quarter. It rejoins the default room (`MCHATROOM`, `#msxpi` unless you changed it); other rooms you had joined are not rejoined, and the scrollback from before the game is gone. Through the MSXPi the Pi keeps the chat and nothing is lost.

Over InterNestor a game packet goes out every third frame, which adds one frame of input delay and makes the game about 10 % faster; the MSXPi and hardware stacks send every frame.

## 4.9 BASIC programs

| Program | What it does |
|---|---|
| `API.BAS` | Template and examples for developers (section 6) |
| `IRC.BAS` | IRC client: nick and server from IRCNICK, IRCADDR, IRCPORT |
| `WEATHER.BAS, DOLAR.BAS` | Weather and exchange-rate clients (from Retropix Brazil) |
| `STOCKS.BAS` | Stock quotes; needs the API keys in msxpi.ini |
| `TESTDIR.BAS` | Directory listing test |

When `CALL MSXPIVER` is not available (no ROM in the interface), the programs `BLOAD"MSXPIEXT.BIN",R` first. This loads the BASIC extension into page-1 RAM and adds `CALL MSXPIVER` and `CALL MSXPI`.

## 4.10 Dealing with errors and instabilities

Because the interface, the Pi and the MSX are all involved, MSXPi may fail. Electrical noise and bus quality vary between MSX models; the interface is very stable on clones such as the Zemmix and on machines with recent SoC chips, and can be less stable on older machines. The transfer routines detect errors and retry on their own, but sometimes a transfer fails and the recovery mode starts.

### 4.10.1 Recovery with CTRL+ESC

Hold CTRL and press ESC to abort an operation or to get the prompt back. ESC alone is passed to the program as an ordinary key. The server stops the transfer and returns to command mode; this takes up to five seconds, so wait before retrying. If the result is "Connection Error" again after a retry, the Pi needs inspection: log in over SSH or with a keyboard and monitor and run the server by hand (`cd /home/pi/msxpi && python3 msxpi-server.py`) to read its messages.

### 4.10.2 Checking what is alive

1. `pver` (DOS) or `CALL MSXPIVER` (BASIC) - is the ROM present?

2. `CALL MSXPI("1,C000,ver")` or `pver` - does the server answer? A "Build ID" line and a "Server" line mean both ends work.
3. The interface LED on: the server is running.
4. A mismatch between the ROM, server and `.COM` versions is the most common cause of "Connection Error" after an update.

### 4.10.3 Failed MSX-DOS 1 boot with a working MSXPi

If `pver` works but MSX-DOS 1 will not boot from the disk image, the image may be corrupt. Options, in order:

- Press the MSX RESET button if there is one: the Pi keeps running and the boot may succeed.
- Without a reset button, from BASIC: `DEFUSR0=0 : A=USR(0)`.
- Test the DOS with `FILES`, then with `POKE &HF346,1` and `FILES`, and finally `POKE -1,170` and `FILES`.
- As a last resort switch the MSX off and on. Replace `msxpiboot.dsk` (the updater `msxpiupd.bat` does it when you boot from another interface).

### 4.10.4 Failed boot with a failing MSXPi

Log in to the Pi over SSH and run the server by hand to read its messages. If the disk image was corrupted by repeated power cuts, copy a fresh one from a PC:

```
scp msxpiboot.dsk pi@raspberrypi:msxpi/disks/
```

The best prevention is `p shut` before every power-off.

### 4.10.5 Older boards

Boards built for v0.8.2 and v1.0 have no /WAIT connection and run polled only; `RPI_SHUTDOWN=none` (no push-button) and, on the v0.8.2 board, `GPIO_CS_SETUP_NS=1000` are required, and the EPROM /OE jumper must be on CS1, never CS12. All of it is in `documents/Legacy Support.odt`.

# 5 Developers Guide

MSXPi programs have two halves: a **client** on the MSX (in BASIC, C or Assembly) and, when new functionality is needed, a **Python function** on the server. The block protocol of section 2.3 is hidden by the BIOS routines of each language, so a client is normally: send a command, receive and use the reply.

## 5.1 The programming model

1. The client sends a **command** (a text of up to BLKSIZE bytes). The first word is the name of a Python function in `msxpi-server.py`; everything after it is the parameter string.
2. The server function does its work and answers with `sendmultiblock(data)`. Large data is split into blocks automatically: the return code is RC_READY on all but the last block.
3. The client reads blocks until a return code other than RC_READY arrives. RC_SUCCESS means done; RC_FAILED means the block holds an error message; RC_TERMINATE ends the conversation.
4. For conversations (upload, paging, state) the client calls the same server function several times with sub-commands, as `pcopy` does.

## 5.2 Adding a command to the server

The template in the server is the whole recipe. The first word of the command is looked up in the COMMANDS table at the top of msxpi-server.py, so the function name is the command name (in lower case) and a new function must be added to that table. A function that returns a str or bytes has its return value sent as the reply:

```
def template(parms = None):
    # If your MSX command send parameters, we go read them:
    if parms == None or parms == "":
        rc = sendmultiblock("This command requires a parameter".encode())
        return

    response = f"Response from MSXPi: I received parameter '{parms}'"
    rc = sendmultiblock(response.encode())
   return
```

To create hello, copy the template, rename it hello, add hello to COMMANDS in msxpi-server.py, restart the server (p restart) and call it from any client: p hello world, CALL MSXPI("1,C000,hello world"), or a client of your own. Guidelines:

- Always reply, even on error - a client waits for the reply. Use `sendmultiblock(msg.encode())`; for a failure use `sendmultiblock(msg.encode(), RC_FAILED)`.
- Keep the command name to letters; it is lower-cased. `set` is mapped to `pset` because `set` is a Linux command.
- Long-running functions may read more from the MSX (see how the pcopy and disk functions in msxpi_cmd_files.py and msxpi_cmd_disk.py receive blocks) and send further blocks; the MSX must do the mirror image.

- Read settings with `getMSXPiVar("NAME")` and store them with `p set`, so they survive in msxpi.ini.
- Anything that only makes sense on a Raspberry Pi should check `hostType == "RaspberryPi"` and answer "Command not supported by this platform" otherwise, as `play` does.

## 5.3 Tools and build

- **C clients** are compiled with SDCC and the Fusion-C headers, linked against `C-common/lib/msxpi-bios.lib`, with a custom MSX-DOS start-up file.
- **Assembly clients and the ROM** are assembled with zmac / sjasmplus (see the header of `software/build`); the transfer routines in `asm-common/transport/payload.z80` are generated for the different consumers by `generate.py`.
- **`software/build`** builds the client commands, the ROM, the LOADROM patch, the openMSX extension and the disk images, and bumps the BuildId of each component when asked.
- **BASIC** needs nothing but the ROM (or MSXPIEXT.BIN).

## 5.4 Testing under openMSX

Develop and test with the MSXPi extension of openMSX: the emulated device implements the CPLD including /WAIT bursts (with fault injection for a low RPI_READY), the server runs on your PC, and its console shows every command received (-> dir /home/pi). What works there works on the hardware. Server unit tests are in software/Server/Python/tests; the UNAPI test harness (openMSX driven) is in software/UNAPI/harness.

# 6 Developing in BASIC

BASIC programs talk to MSXPi with one statement, `CALL MSXPI`, provided by the ROM or by `MSXPIEXT.BIN`. (The old `CALL MSXPISEND` and `CALL MSXPIRECV` no longer exist; everything is done with commands and replies.) `CALL MSXPIVER` prints the BIOS version and is the way to test that the BIOS is present.

## 6.1 CALL MSXPI

```
CALL MSXPI("<stdout>,<buffer>,<command> <parameters>")
```

| Part | Meaning |
|---|---|
| `stdout` | 0: discard the reply. 1: print the reply on the screen. 2: store the reply in memory, do not print. |
| `buffer` | Four hex digits, for example C000. Where the reply is stored (mode 2), and where the return code is left. |
| `command` | Name of the server function, with its parameters, exactly as in `p <command>`. |

```
CALL MSXPI("0,C000,set")                    ' run, ignore the reply
CALL MSXPI("1,C000,dir /home/pi/msxpi")      ' print the listing
CALL MSXPI("2,D000,dir /etc")                ' store the listing at D000
```

In mode 2 all blocks of the reply are stored one after the other after a three-byte header:

| Address | Content |
|---|---|
| `buffer + 0` | Return code of the last block (see Appendix A): &HE0 success, &HE7 failed, &HE8 connection error |
| `buffer + 1, + 2` | Total size of the data, low byte then high byte |
| `buffer + 3 ...` | The data (a text reply is plain ASCII) |

```
SIZE = PEEK(&HD001) + 256 * PEEK(&HD002)
RC = PEEK(&HD000)
FOR I = 0 TO SIZE - 1 : PRINT CHR$(PEEK(&HD003 + I)); : NEXT
```

> **Buffer safety:** The buffer must hold the whole reply plus three bytes, and MSXPi does not check: a larger reply overwrites memory above the buffer. Lower the top of BASIC memory first, as API.BAS does with `CLEAR 200,(PEEK(&HFC4A)+256*PEEK(&HFC4B))-520`, and choose a buffer address inside the area you reserved. For replies of unknown size use mode 1, or read the reply block by block with the API routines below.

If the link fails, the return code in the buffer is &HE8 (connection error) and the size is zero, so a program can always test `PEEK(buffer)`.

## 6.2 The BASIC API: API.BAS

`API.BAS` wraps CALL MSXPI in two subroutines and shows four examples. Copy the file, keep lines 50 to 100 and 9999 onwards, and write your program from line 101. Lines 60 to 61 reserve 520 bytes at the top of memory for the buffer (BLKSIZE plus the header) and put the buffer address in BF. Lines 70

to 90 are shipped as comments: remove the leading apostrophes to load MSXPIEXT.BIN automatically when `CALL MSXPIVER` fails.

| Routine | Line | Inputs | Result |
|---|---|---|---|
| SENDCOMMAND | 10000 | A = stdout (0, 1, 2)<br>B = buffer address (&HC000)<br>C$ = command<br>D$ = parameters | RC = return code, -1 on error |
| PRINTSTDOUT | 11000 | A = 1 print, 0 return in C$<br>B = buffer address<br>C = characters to return (A=0) | RC = &HE0 more data, &HED end |

**Example 1** - a DIR of the Pi printed directly (from API.BAS lines 110 to 153):

```
110 A=1                     ' stdout: print to screen
 120 B=&HC000                 ' buffer address
 130 C$="dir"                 ' command
 140 D$="/home/pi/msxpi"      ' parameters
 150 PRINT:GOSUB 10000        ' SENDCOMMAND
153 PRINT:PRINT"finished:";HEX$(RC)
```

**Example 2** - keep the reply in memory and print it with PRINTSTDOUT (lines 210 to 260):

```
210 A=2                     ' stdout: save to buffer only
 220 B=&HC000
 230 C$="dir"
 240 D$="/home/pi/msxpi"
 250 PRINT:GOSUB 10000
 253 IF RC=-1 THEN GOTO 260    ' connection error
255 A=1:GOSUB 11000          ' PRINTSTDOUT: print the buffer
```

**Example 3** - read the reply into a string variable, 40 characters at a time (lines 310 to 365):

```
310 A=2:B=&HC000:C=40         ' C = characters per string
 330 C$="dir":D$="/home/pi/msxpi"
 350 PRINT:GOSUB 10000
 353 IF RC=-1 THEN GOTO 365
 355 A=0:GOSUB 11000          ' next 40 characters into C$
 360 PRINT(C$);:IF RC<>&HED AND RC<>&HE7 THEN GOTO 355
365 PRINT:PRINT"finished:";HEX$(RC)
```

**Example 4** - a custom server command, `template`, with parameters (lines 410 to 495) - the same server function shown in section 5.2:

```
410 A=1:B=&HC000
 430 C$="template":D$="par1 par2 par3"
450 PRINT:GOSUB 10000
```

SENDCOMMAND itself is only a call to the ROM. This is the whole routine (lines 10002 to 10010):

```
10002 IFA=0THENCALLMSXPI("0,"+HEX$(B)+","+C$+" "+D$):GOTO 10010
 10003 IFA=1THENCALLMSXPI("1,"+HEX$(B)+","+C$+" "+D$):GOTO 10010
 10004 CALLMSXPI("2,"+HEX$(B)+","+C$+" "+D$)
10010 RC=PEEK(B):RETURN
```

Note that `HEX$(B)` builds the four-digit buffer address, and that the return code of the whole exchange is the first byte of the buffer.

> **Obsolete:** The routines MSXPISEND (line 12000), MSXPIRECV (13000) and STORESTR (14000) in API.BAS are marked discontinued and refer to CALL commands that no longer exist. Do not use them.

## 6.3 Tips for BASIC programs

- Test for the BIOS first: `ON ERROR GOTO 90 : CALL MSXPIVER : GOTO 100`, and at line 90 `BLOAD"MSXPIEXT.BIN",R`, as API.BAS does.
- Always test RC before using the data. RC &HE8 is a communication problem; RC &HE7 means the buffer holds an error text.
- Programs that must also run under openMSX should not assume 40 columns; the bundled programs default to 80 (`var WIDTH=80`).
- IRC.BAS shows a program that keeps state on the server across many CALL MSXPI calls instead of holding the server for a single long command.

# 7 Developing in C

C is the language of the `p` commands themselves: `p.com`, `pcopy.com`, `pver.com` and `msxarch.com` are C programs. The C BIOS wraps the transport routines in a small API, declared in `software/C-common/header/msxpi.h` and implemented in `software/C-common/lib/msxpi-bios.c` (compiled to `msxpi-bios.lib`). Programs are built with SDCC and the Fusion-C headers for MSX-DOS; the library is linked with a custom MSX-DOS start-up file (`--no-std-crt0`).

## 7.1 A minimal program

`Client/src/templatc.c` is the template for a new C command. It sends the command `template` with the DOS command-line parameters appended, and prints the reply:

```
#include <stdint.h>
#include <stdbool.h>
#include <stddef.h>
#include "../../../../../MSX/MSX-C/WorkingFolder/fusion-c/header/msx_fusion.h"
#include "../../C-common/header/msxpi.h"

int main(void)
{
    uint8_t rc = SendCommandToMSXPi("template", true);
    if (rc == RC_SUCCESS || rc == RC_FAILED) {
        uint8_t* buffer = (uint8_t*)(get_buffer_ptr() + 100);
        printstdout(buffer, MAXBUFSIZE);
    }
    return 0;
}
```

That is the pattern for the vast majority of commands: **send the command, then print the reply**. `template A B` sends "template A B", the server function `template("A B")` runs, and its reply is printed.

## 7.2 API reference

| Function | Purpose |
|---|---|
| `uint8_t SendCommandToMSXPi(const char* cmd, bool appendDOSParameters)` | Builds "cmd" (plus the DOS command line when the flag is true; a leading space is inserted) in the transfer buffer and sends it as a block. Returns RC_SUCCESS, or an error code. With an empty `cmd` it sends only the DOS command line, which is how `p.com` works: `p dir /` sends "dir /". |
| `uint8_t printstdout(uint8_t* buffer, uint16_t maxbufsize)` | Receives every block of the reply and prints it as text, until the final block. Does the handshake itself. Returns the final return code. |
| `uint8_t PerformHandshake(uint16_t msx_blocksize)` | The READY / READY_ACK handshake plus the announcement of the largest block the MSX accepts. Call it before RECVDATA_ONEBLOCK. |
| `uint8_t RECVDATA_ONEBLOCK(uint8_t* dest, uint16_t* size, uint16_t msx_blocksize)` | Receives exactly one block into `dest`, with checksum verification and retries; `size` receives the payload length. Returns RC_READY (more blocks follow), RC_SUCCESS (last block) or an error. |
| `uint8_t RECVDATA(uint8_t* dest, uint16_t* size, uint16_t* maxbufsize)` | Receives a complete multi-block reply into `dest` (handshake included). `size` receives the total length. |

| Function | Purpose |
|---|---|
| `uint8_t SENDDATA2(uint8_t* src, uint16_t size, uint16_t* maxbufsize)` | Sends one block (handshake, header, payload, checksum, retries). Used by SendCommandToMSXPi and for uploads. |
| `uint8_t parseConnError(uint8_t rc)` | Prints a message for a transport error code ("Checksum mismatch", "Error connecting to MSXPi server", ...) and returns the code. |
| `char* GetCmdLineParameters(void)` | The DOS command line, trimmed of leading spaces. |
| `uint8_t* get_buffer_ptr(void), uint16_t get_max_buffer_size(void)` | Start of free RAM after the program (the heap top) and how much is left before the stack. |
| `void msxpi_link_claim(void), void msxpi_link_release(void)` | Hold the MSXPi link for the whole of an exchange (below). |
| `uint8_t msxpi_exchange(cmd, appendTail, buffer, maxbufsize)` | Claim, send `cmd`, print the reply, release - in one call, with a single exit so the link cannot be left claimed. |
| `uint8_t PIREADBYTE(uint8_t*), PIWRITEBYTE(uint8_t), CHKPIRDY(void)` | Byte-level primitives on the ports. Normally not needed. |
| `pprintf(text, value), pprints(text, text), FastPrint(text)` | Small print helpers that avoid the weight of printf. |

The constants in `msxpi.h`: ports CONTROL_PORT1 0x56, CONTROL_PORT2 0x57, DATA_PORT1 0x5A; BLKSIZE 512; MAXBUFSIZE 8192; BUFADDRESS 0xC000; GLOBALRETRIES 1 and MAX_BLOCK_RETRIES 3; the READY/ACK bytes; and the RC_ return codes of Appendix A.

## 7.3 Example: pver.c - talk to the hardware and the server

`pver` reads the CPLD build ID directly from port $57, probes for /WAIT mode, then asks the server for its version with the same request `p ver` makes. It holds the link while it does so:

```
int main(void)
{
    uint8_t raw, rc;

    raw = InPort(CONTROL_PORT2);
    Print("Port $57: "); PrintHexByte(raw); Print("\r\n");

    if (raw >= 0xFE) {
        /* No MSXPi CPLD answered: nothing to ask the server through. */
        Print("Board   : no MSXPi found\r\n");
        return 0;
    }
    print_board(raw);

    msxpi_link_claim();
    rc = SendCommandToMSXPi("ver", false);
    if (parseConnError(rc) == RC_SUCCESS || rc == RC_FAILED || rc == RC_BUFOVFLW) {
        printstdout((uint8_t *)(get_buffer_ptr() + 100), MAXBUFSIZE);
    } else {
        Print("Server  : connection error\r\n");
    }
    msxpi_link_release();
    return 0;
}
```

`print_board()` shows how /WAIT support is detected: switch the mode on, see whether the port changes, switch it off again:

```
OutPort(CONTROL_PORT2, 0x01);          /* try /WAIT mode on  */
probe = InPort(CONTROL_PORT2);
OutPort(CONTROL_PORT2, 0x00);           /* and off again      */
...
Print(probe != (raw & 0x7F) ? "yes (burst)" : "no (polled)");
```

## 7.4 Example: pcopy.c - a conversation with several blocks

`pcopy` shows the second common shape: the client calls one server function with **sub-commands** (`pcopy init`, `pcopy readblock`, `pcopy putclose`, ...) and receives one block per request, writing it to disk between requests. The receive loop of a Pi-to-MSX copy:

```
while (1) {
    /* Request next block from server */
    rc = SendCommandToMSXPi("pcopy readblock", false);
    if (rc != RC_SUCCESS) {
        Print("\r\nConnection error during read.\r\n");
        fcb_close(&file);
        return parseConnError(rc);
    }

    rc = PerformHandshake(maxbufsize);
    if (rc != RC_SUCCESS) { /* handshake error, close, return */ }

    Print(".");
    rc = RECVDATA_ONEBLOCK(buffer, &block_size, maxbufsize);

    /* A failed receive can leave block_size and the buffer partly filled:
       never commit that data to disk. */
    if (rc != RC_SUCCESS && rc != RC_READY) {
        Print("\r\nTransfer aborted by server.\r\n");
        fcb_close(&file);
        return rc;
    }

    if (block_size > 0 && fcb_write(&file, buffer, block_size) != 0) { /* disk error */ }

    if (rc == RC_SUCCESS) break;      /* final block received */
}
```

The pattern to remember: **send a request, `PerformHandshake`, `RECVDATA_ONEBLOCK`, act on the return code** - RC_READY means come back for more, RC_SUCCESS means that was the last one, anything else is an error. Uploading (MSX to Pi) uses `SENDDATA2` in the same way.

## 7.5 Example: msxarch.c - reading structured data

When the reply is binary rather than text, read it with `RECVDATA` into your own buffer and pick it apart. `msxarch` does this for the 16-byte ROM header the server sends before every ROM (Figure 6):

```
uint8_t readRomHeader(RomHeader* hdr, char* reason, uint16_t reasonBufSize) {
    uint8_t  buf[ROM_MSG_MAX];
    uint16_t replySize = 0;
    uint16_t maxbuf = ROM_MSG_MAX;
    uint8_t  rc = RECVDATA(buf, &replySize, &maxbuf);

    if ((rc != RC_SUCCESS && rc != RC_TERMINATE) ||
        replySize < ROM_HEADER_SIZE || buf[0] != ROM_HEADER_MAGIC)
        return RC_FAILED;

    hdr->mapperType = buf[2];
    hdr->bankSizeKB = buf[3];
    hdr->bankCount  = (uint16_t)buf[4] | ((uint16_t)buf[5] << 8);
   ...
```

## 7.6 The shared link and the Ethernet UNAPI

When the Ethernet UNAPI is installed, an interrupt routine polls the Pi for incoming frames. If it transmitted in the middle of a command exchange, both streams would be corrupted. A program that talks to MSXPi for a long time, or that mixes several transport calls, should bracket the exchange with

`msxpi_link_claim()` and `msxpi_link_release()`, or use `msxpi_exchange()`, which does so with one entry and one exit. Both are no-ops when no Ethernet UNAPI is installed.

## 7.7 Memory and buffers

- The transfer buffer is `char buffer[MAXBUFSIZE]` pinned at BUFADDRESS ($C000) in `SendCommandToMSXPi`. It is the largest consumer of data memory; programs that switch mapper segments into page 2 (msxarch) relocate it deliberately.
- Replies are normally received into memory obtained from `get_buffer_ptr()` (the top of the program's heap); `get_max_buffer_size()` tells how much room there is before the stack. `p /m` shows these numbers for `p.com`.
- A reply larger than `maxbufsize` returns RC_BUFOVFLW; the part that fits has been received. Pass a buffer size that fits your program.

## 7.8 Error handling

- Transport failures (after the retries) surface as return codes: RC_CHKSUM_ERR, RC_HANDSHAKEERR, RC_CONNERR, RC_BUFOVFLW. `parseConnError()` prints a message for them.
- RC_FAILED is not a transport failure: the server ran your function and reported an error in the reply text - print it with `printstdout()`.
- CTRL+ESC pressed by the user is detected inside the transport and returns an error code; do not retry immediately, the server needs about five seconds to resynchronise.

# 8 Developing in Assembly

The assembly BIOS is `software/asm-common/include/msxpi_bios.asm`, together with `include.asm` (constants and return codes) and a `putchar_*.asm` file that supplies `PUTCHAR` for the environment (MSX-DOS or BASIC). The same code is linked into the ROM, the disk driver, the BASIC extension and the `.COM` programs, so what is described here is exactly what the rest of MSXPi uses.

## 8.1 Skeleton of a .COM program

`Client/src/template.com.asm` is the full template. It is an MSX-DOS program (origin $0100) that copies the DOS command line after the command name, sends it, and prints the reply:

```
       org     $0100

MAIN:
        call    resetMSXPI          ; clear the transfer state

; Append the DOS command-line parameters (at $80) to the command text
        ld      hl,0x80
        ld      a,(hl)
        or      a
        jr      z,sendCmd
        ld      c,a
        ld      b,0
        inc     hl
        ld      de,parms
        ldir
        xor     a
        ld      (de),a

sendCmd:
        ld      de,command
        call    SendCommandToMSXPi
        jr      c,PRINTPIERR

        ld      de,buf
        ld      bc,BLKSIZE
        call    CLEARBUF
        ld      de,buf
        ld      bc,BLKSIZE
        call    PRINTPISTDOUT
        call    PRINTNLINE
        ret

PRINTPIERR:
        ld      hl,PICOMMERR
        jp      PRINT
PICOMMERR:  db  "Communication Error",13,10,0

; Core MSXPi APIs / BIOS routines
        INCLUDE "include.asm"
        INCLUDE "putchar_clients.asm"
        INCLUDE "msxpi_bios.asm"

command: db "template "
parms:  db 0
        ds 255
buf:    equ $
        ds BLKSIZE
       db 0
```

The command string must be terminated with zero. It is sent as a block, so its length is limited only by the transfer buffer; the server splits it at the first space. A `.COM` program has to define which scratch

area the BIOS may use for its receive state: linked into a plain `.COM` it needs `MSXPI_RAM_STASH equ 1` before the include (the disk driver defines `MSXPI_DRIVER`, and neither symbol defined is a build error - a wrong choice fails silently at run time).

## 8.2 BIOS routines

All routines preserve the registers not listed. "C" and "NC" are the carry flag on return.

| Routine | In | Out | Description |
|---|---|---|---|
| `resetMSXPI` | - | - | OUT ($56),$FF: reset the CPLD transfer state (and /WAIT mode). Call at the start of a program. Not assembled into the ROM. |
| `CHKPIRDY` | - | C: CTRL+ESC pressed; NC: ready | Waits until the Pi is ready (port $56 = 0), sampling CTRL+ESC from the key matrix. |
| `PIWRITEBYTE` | A = byte | C on CTRL+ESC | Waits for ready and writes A to port $5A. |
| `PIREADBYTE` | - | A = byte; C on CTRL+ESC | Starts a transfer, waits for it and reads port $5A. |
| `SendCommandToMSXPi` | DE = zero-terminated command | NC ok; C error; DE advanced | Sends the command as one block (a wrapper on SENDDATA). An empty string returns C. |
| `SENDDATA` | DE = source, BC = size (at most MAXBUFSIZE) | NC ok, DE = source + size; C error | One block: handshake, header, payload, checksum, up to GLOBALRETRIES retries. |
| `PerformHandshake` | A = block index, DE = destination, BC = block size the MSX accepts | NC ok; C error; A unchanged | READY / READY_ACK, then announces BC. Required before RECVDATA_ONEBLOCK. Reports its own carry (v1.6 fix). |
| `RECVDATA_ONEBLOCK` | A = expected block index, DE = destination, BC = block size | A = header return code, BC = payload length, DE = destination + length; C on error | Receives one block with checksum check and retries. The payload is written to memory only (no header); use the returned code to decide whether another block follows (RC_READY). |
| `PRINTPISTDOUT` | DE = buffer, BC = its size | A = last return code; C on error | Handshake, then receives every block and prints it as text. Terminates the text with zero. |
| `STDOUTTONULL` | DE = buffer, BC = its size | A, C as above | Same, but discards the reply. |
| `PRINT` | HL = zero-terminated text | - | Prints through PUTCHAR. |
| `PRINTNLINE` | - | - | Prints CR LF. |
| `PRINTNUMBER` | see source | - | Prints a number. |
| `STRTOHEX, ATOHEX` | see source | - | ASCII hexadecimal digits to binary. |
| `CLEARBUF` | DE = buffer, BC = size | - | Fills a buffer with zeroes. |
| `STORE_BYTE` | A = byte, DE = address | - | Stores A at (DE) correctly even when DE is in page 1 (banked ROM). |

Register and stack usage matters in a few places: `RECVDATA_ONEBLOCK` uses IX and IY internally and the alternate register set (EXX) in its payload loop, and interrupts are managed by the routine (the payload loops run with them disabled). Do not call it from an interrupt handler.

## 8.3 Example: receiving a reply block by block

`PRINTPISTDOUT` is itself a loop over `RECVDATA_ONEBLOCK`, and it is the model for any program that needs the data rather than printing it. This is the core of the routine (`msxpi_bios.asm`):

```
PRINTPISTDOUT:
    call    PerformHandshake    ; required before RECVDATA_ONEBLOCK
    xor     a                   ; block number = 0
PRINTPISTDOUT0:
    push    af                  ; save block number
    push    bc                  ; maxbufsize expected
    push    de                  ; save buffer address
    call    RECVDATA_ONEBLOCK   ; read one block
    ld      l,a                 ; save return code
    ld      a,TEXTTERMINATOR
    ld      (de),a              ; terminate the text
    ...
    call    PRINT
    ...
    cp      RC_READY            ; another block?
    ret     nz                  ; no: return
    ...                         ; restore, next block number
    inc     a
   jr      PRINTPISTDOUT0
```

To collect a reply in memory instead, copy this loop and move DE forward by BC after each block. The BASIC handler (`msxpi_call.asm`, `CALL_MSXPISAVEDATA`) does that: it accumulates the block lengths in HL and writes the header at the start of the buffer when done.

## 8.4 Example: the BASIC CALL handler

The `CALL MSXPI` statement of the ROM and of MSXPIEXT.BIN is one shared source, `asm-common/include/msxpi_call.asm`. It is a good example of a routine that uses the BIOS from inside a ROM: it evaluates the BASIC string, splits it into stdout mode, buffer and command, sends the command and dispatches on the mode:

```
_MSXPI:
        CALL    EVALTXTPARAM    ; evaluate the text parameter
        PUSH    HL              ; save for BASIC
        CALL    GETSTRPNT       ; address of the parameters
        EX      DE,HL
        CALL    PARSE_CALL_PARAMETERS
        JP      C,CALL_SYNTAX_ERROR
; A = stdout mode, DE = command, HL = buffer address
        ...
        CALL    SendCommandToMSXPi
        JR      NC,CALL_SUCCESS
        ...                     ; connection error: [0]=RC_CONNERR, size 0
CALL_SUCCESS:
        cp      '0'
        jr      z,CALL_DiscardResponse   ; STDOUTTONULL
        CP      '2'
        JR      Z,CALL_MSXPISAVEDATA     ; store in the buffer
        ld      bc,MAXBUFSIZE            ; mode 1: print
        xor     a
       call    PRINTPISTDOUT
```

## 8.5 Example: the disk driver

The MSX-DOS 1 disk driver (built for Nextor coexistence too) in `ROM/src/MSX-DOS/msxpi-driver.mac` is the most demanding user of the BIOS: DSKIO sends `dskior` or `dskiow` with the sector number and count, then moves 512-byte payloads. It uses `MSXPI_DRIVER`, keeps its receive state in the driver work area obtained with GETWRK, transfers straight to the caller's buffer when it is outside page 1 (and through the kernel XFER routine when it is not), and asks for /WAIT bursts (INIR) only after wait mode has been proved to work. It is the reference for anything that needs speed.

## 8.6 Other useful sources

- `Client/src/at28c256.com.asm` - a complete tool: slot detection, page switching and reading the EEPROM from MSX-DOS.
- `asm-common/transport/payload.z80` and `generate.py` - the burst and polled payload loops, generated for each consumer.
- `UNAPI/src` - the Ethernet UNAPI driver (`ethcore.asm`, `ethops.asm`, `ethtrans.asm`) and test clients.

# 9 Appendices

## 9.1 Appendix A: Return codes and protocol constants

| Value | Name | Meaning |
|---|---|---|
| $A0 | READY_ACK | Handshake acknowledgement from the receiver |
| $A1 | SENDNEXT | Request the next chunk |
| $A2 | ENDTRANSFER | End of transfer |
| $AA | READY | Handshake: I am ready |
| $AD | RC_CHKSUM_ERR | Checksum still wrong after all retries |
| $AE | BUSY / WAIT | Busy marker |
| $E0 | RC_SUCCESS | Success; last block |
| $E1 | RC_INVALIDCOMMAND | Invalid or missing command or parameters |
| $E2 | RC_ESCPRESSED | CTRL+ESC pressed |
| $E3 | RC_BUFOVFLW | Reply larger than the buffer |
| $E4 | RC_INVALIDDATASIZE | Invalid data size |
| $E5 | RC_HANDSHAKEERR | Handshake failed |
| $E6 | RC_FILENOTFOUND | File not found on the Pi |
| $E7 | RC_FAILED | Server function failed; the block holds an error text |
| $E8 | RC_CONNERR | Connection error (set by the MSX) |
| $E9 | RC_WAIT | Wait |
| $EA | RC_READY | Success; more blocks follow |
| $EB | RC_SUCCNOSTD | Success, no standard output |
| $EC | RC_FAILNOSTD | Failure, no standard output |
| $ED | RC_TERMINATE | The server ends the conversation; nothing follows |
| $EE | RC_UNEXPECTEDDATA | Unexpected data received |
| $EF | RC_UNDEFINED | Undefined |

Sizes: BLKSIZE = 512 bytes per block; MAXBUFSIZE = 8192 bytes; retries per block MAX_BLOCK_RETRIES = 3. Bit 15 of the length (or of the block size the MSX announces) is the /WAIT burst flag.

## 9.2 Appendix B: msxpi.ini variables

The file is a list of `var NAME=value` lines (comments start with `#`) in the MSXPi home directory. `p set NAME value` changes a value and saves the file.

| Variable | Default / example | Purpose |
|---|---|---|
| PATH | /home/pi/msxpi | Current path for p dir / pcopy |
| DriveA, DriveB | disks/msxpiboot.dsk, disks/tools.dsk | Disk images for drives A: and B: |
| DriveM, DriveR1, DriveR2 | ftp://192.168.1.100, msxarchive.nl | The m:, r1: and r2: virtual drives |

| Variable | Default / example | Purpose |
|---|---|---|
| | MSX1 / MSX2 | |
| WIDTH | 80 | Screen width for the bundled programs |
| WIFISSID, WIFIPWD, WIFICOUNTRY | -, -, GB | Used by `p wifi set` |
| DSKTMPL | disks/blank.dsk | Template for new disk images |
| IRCNICK, IRCADDR, IRCPORT | msxpi, chat.freenode.net, 6667 | IRC client |
| OPENAIKEY, OPENAIMODEL | -, gpt-4o-mini | ChatGPT (`p chatgpt`) |
| RAPIDAPIKEY, RAPIDAPIHOST, FINNHUBKEY, TWELVEDATAKEY, ALPHAVANTAGEKEY | placeholders | Stock quote providers (STOCKS.BAS). No key ships with MSXPi. |
| SPI_HW | False | Use the Pi hardware SPI instead of GPIO bit-banging |
| SPI_CS, SPI_SCLK, SPI_MOSI, SPI_MISO, RPI_READY | 21, 20, 16, 12, 25 | GPIO numbers of the link (Appendix C) |
| RPI_SHUTDOWN | 26 (unset), or none | GPIO of the shutdown/reboot button; `none` disables it (boards without the button) |
| GPIO_CS_SETUP_NS | 0 (unset) | Settling time between CS low and the first clock edge; 1000 on the v0.8.2 board |
| ROMDB | URL or path | Optional ROM database used by msxarch to name and classify ROMs |

Environment: `MSXPI_NATIVE_GPIO=1` (on by default in msxpi-monitor and the systemd units) enables the native GPIO engine; `MSXPI_GPIO_CS_SETUP_NS` overrides GPIO_CS_SETUP_NS.

## 9.3 Appendix C: GPIO pin assignments

The default wiring between the interface and the Raspberry Pi. To connect an MSXPi to a Raspberry Pi 2/3/4 by wires, use the same pins; a wired connection can be unstable.

| MSXPi signal | Raspberry Pi GPIO | Optional GPIO (PCB v1.1 Rev.0 jumpers) | Header pin (primary / optional) |
|---|---|---|---|
| CS | 21 | 8 | 40 / 24 |
| CLK | 20 | 11 | 38 / 23 |
| MOSI | 16 | 10 | 36 / 19 |
| MISO | 12 | 9 | 32 / 21 |
| SPI_READY (RPI_READY) | 25 | 24 | 22 / 18 |
| Shutdown button | 26 | - | 37 |
| GND | GND | GND | 6, 9, 14, 20, 25, 30, 34, 39 |

The optional pins, when used, must also be set in msxpi.ini. Pin numbering of the Raspberry Pi header:

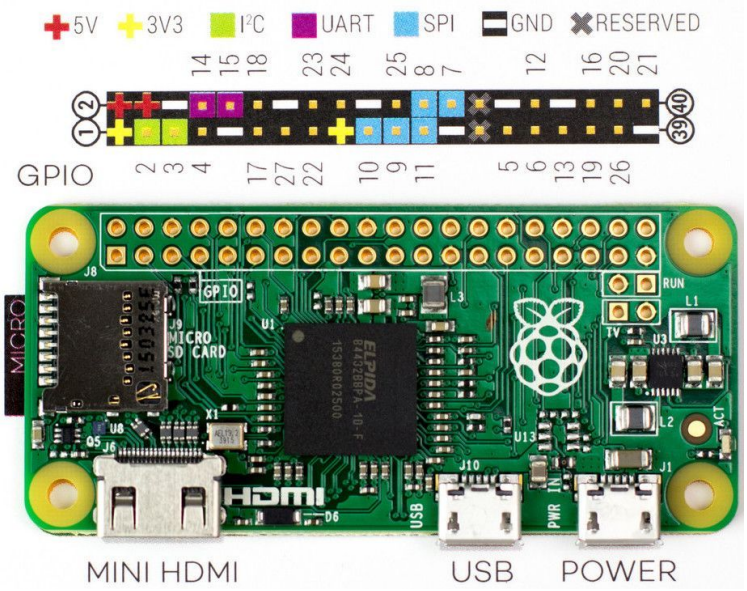



*Figure 11: GPIO numbering, models B+, 2, 3 and Zero*

## 9.4 Appendix D: The MSXPi device for openMSX and the CPLD

The openMSX device (`software/openMSX/src/MSXPiDevice.cc`) is the software counterpart of the VHDL design in `hardware/CPLD_Project`. Both implement the same ports, protocol and control flow, so the same ROM and the same server work with either; they differ in execution model.

| Aspect | CPLD (VHDL) | openMSX device |
|---|---|---|
| Timing | Bit-level, on the rising edge of SPI_SCLK | Byte-level over a TCP socket; asynchronous and buffered |
| Concurrency | Parallel signals reacting to clock edges | A reader thread and queues |
| Bus direction | Drives BUSDIR_n, tristates the data bus | Simulates reads and writes; no electrical contention |
| SPI signals | Drives SPI_MOSI, MISO, CS directly | Emulated through the TCP protocol |
| /WAIT | WAIT_n held while a byte is in flight (v1.6) | Emulated stall of about 20 us per byte, with fault injection for a low RPI_READY |
| Server link | GPIO, 5 wires | TCP, default port 5000 (`msxpiserver_port`), TCP_NODELAY, back-pressure instead of discarding bytes |
| Firmware ID | Read from port $57 | Reports $0E, a v1.6 /WAIT build |

This equivalence lets developers test and debug MSXPi software under openMSX before running it on hardware, and extend the protocol in either domain with confidence. The extension is loaded with `-ext MSXPi`; `MSXPi.xml` places the device on ports $56, $57 and $5A and the 16 KB BIOS ROM at $4000. A variant without the ROM (`MSXPiNoROM`) exists for testing MSXPIEXT.BIN.

## 9.5 Appendix E: Files in software/target

| File | What it is |
|---|---|
| `msxpibios.rom` | The ROM for the EEPROM (16 KB): BIOS, disk driver, Ethernet UNAPI |
| `msxpiext.bin` | MSXPIEXT.BIN: the BASIC extension for interfaces |

| File | What it is |
|---|---|
| | without an EEPROM |
| `p.com, pcopy.com, pver.com, msxarch.com, msxarch.ini, LOADROM.COM, at28c256.com` | The client commands |
| `template.com, templatc.com` | Assembly and C templates for new commands (server function `template`) |
| `INL.CFG, Telnet.com, Telnetf.com, STELNET.COM, CP437.COM` | TCP/IP configuration and clients |
| `skmsx.com` | Utility |
| `API.BAS, IRC.BAS, WEATHER.BAS, DOLAR.BAS, RASTRO.BAS, STOCKS.BAS, TESTDIR.BAS` | BASIC programs |
| `msxpiupd.bat, msxpirfh.bat` | The updater |
| `disks/msxpiboot.dsk, tools.dsk, MSXDOS.SYS, COMMAND.COM` | Disk images for the MSX-DOS 1 boot mode and the kernel files |